# Architectures for Hardware Acceleration

Course Code : UE23EC342BB3

## Course project

Team Members:
Rohan R
Manasa Ajit
Anthariksh K Santusht
Anagha Saraswathy
Neha Prajwal

# Problem Statement

Design and implementation of 5 stage Pipelined RISC-V processor with and without Clock gating in SystemC

# Theory

RISC-V is an open-source ISA (RV32I base) supporting R, I, S, B, U, and J instruction formats. Its simplicity and modularity make it ideal for processor design projects in SystemC or HDL environments

**5-Stage Pipeline**

Pipelining overlaps instruction execution across 5 discrete stages, achieving a theoretical CPI = 1:

- IF
- ID
- EX
- MEM
- WB

Pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) hold inter-stage data, clocked every cycle

# Theory

| | Without Clock Gating | With Clock Gating |
|---|---|---|
| Clock activity | All FFs toggle every cycle | Idle FFs held static |
| Power | Higher dynamic power | Significantly lower |
| Complexity | Simple, no CE logic | Requires hazard-driven CE signals |
| Performance | Unchanged | Unchanged |

# Code: Design - with clock gating

```cpp
#include <systemc.h>
#include <iostream>
#include <cstring>
#include <cstdint>

enum { MEM_WORDS=256, RF_SIZE=32 };

// ================ PIPELINE STRUCTS ================
struct IF_ID  { uint32_t pc=0,inst=0; bool v=false; };
struct ID_EX  { uint32_t pc=0,a=0,b=0,imm=0; uint8_t rs1=0,rs2=0,rd=0; bool v=false; };
struct EX_MEM { uint32_t pc=0,alu=0; uint8_t rd=0; bool v=false; };
struct MEM_WB { uint32_t alu=0; uint8_t rd=0; bool v=false; };

// ================ CPU ================
SC_MODULE(CPU){
    sc_in<bool> clk, rst;

    uint32_t imem[MEM_WORDS];
    uint32_t rf[RF_SIZE];

    IF_ID if_id;
    ID_EX id_ex;
    EX_MEM ex_mem;
    MEM_WB mem_wb;

    uint32_t pc = 0;

    // Signals for waveform
    sc_signal<uint32_t> sig_pc;
    sc_signal<bool> sig_if, sig_id, sig_ex, sig_mem;

    // ================ ALU ================
    uint32_t alu(uint32_t a, uint32_t b){
        return a + b;
    }
```

# Code: Design - with clock gating

```cpp
// ================ PROGRAM ================
void load_program(){
    memset(imem,0,sizeof imem);

    imem[0] = 0x00200093; // addi x1,x0,2
    imem[1] = 0x00300113; // addi x2,x0,3
    imem[2] = 0x002081B3; // add x3,x1,x2
    imem[3] = 0x00418213; // addi x4,x3,4
    imem[4] = 0x002202B3; // add x5,x4,x2
    imem[5] = 0x00000000; // stop
}

// ================ PIPELINE ================
void tick(){
    if(rst.read()){
        pc = 0;
        memset(rf,0,sizeof rf);
        if_id = {}; id_ex = {}; ex_mem = {}; mem_wb = {};
        return;
    }

    // ===== CLOCK GATING ENABLES =====
    bool en_if  = true;
    bool en_id  = if_id.v;
    bool en_ex  = id_ex.v;
    bool en_mem = ex_mem.v;
    bool en_wb  = mem_wb.v;

    // ================ WB ================
    if(en_wb && mem_wb.rd){
        rf[mem_wb.rd] = mem_wb.alu;
    }

    // ================ EX ================
    EX_MEM ex_mem_n = {};
    if(en_ex){
        uint32_t res = alu(id_ex.a, id_ex.b);
        ex_mem_n = {id_ex.pc, res, id_ex.rd, true};
    }
```

```cpp
// ================ MEM ================
MEM_WB mem_wb_n = {};
if(en_mem){
    mem_wb_n = {ex_mem.alu, ex_mem.rd, true};
}

// ================ ID ================
ID_EX id_ex_n = {};
if(en_id){
    uint32_t inst = if_id.inst;

    uint8_t opcode = inst & 0x7F;
    uint8_t rs1 = (inst >> 15) & 0x1F;
    uint8_t rs2 = (inst >> 20) & 0x1F;
    uint8_t rd  = (inst >> 7)  & 0x1F;
    int32_t imm = (int32_t)inst >> 20;

    uint32_t a = rf[rs1];
    uint32_t b = (opcode == 0x13) ? imm : rf[rs2]; // ADDI vs ADD

    id_ex_n = {if_id.pc, a, b,(uint32_t)imm, rs1, rs2, rd, true};
}

// ================ IF ================
IF_ID if_id_n = {};
if(en_if){
    uint32_t instr = imem[pc >> 2];

    if(instr != 0){
        if_id_n = {pc, instr, true};
        pc += 4;
    }
}

// ================ UPDATE ================
if_id = if_id_n;
id_ex = id_ex_n;
ex_mem = ex_mem_n;
mem_wb = mem_wb_n;
```

# Code: Design - with clock gating

```
        // ================= SIGNALS =================
        sig_pc.write(pc);
        sig_if.write(if_id.v);
        sig_id.write(id_ex.v);
        sig_ex.write(ex_mem.v);
        sig_mem.write(mem_wb.v);

        // ================= DEBUG =================
        std::cout << "PC=" << pc
                  << " IF=" << if_id.v
                  << " ID=" << id_ex.v
                  << " EX=" << ex_mem.v
                  << " MEM=" << mem_wb.v
                  << std::endl;

        // ================ STOP ================
        if(!if_id.v && !id_ex.v && !ex_mem.v && !mem_wb.v){
            sc_stop();
        }
    }

    SC_CTOR(CPU){
        load_program();
        SC_METHOD(tick);
        sensitive << clk.pos();
    }
};
```

# Code: Testbench - with clock gating

```
// ================= TESTBENCH =================
SC_MODULE(TB){
    sc_signal<bool> clk, rst;
    CPU cpu;

    void run(){
        rst = 1;
        clk = 0;
        wait(1, SC_NS);

        rst = 0;

        for(int i=0;i<50;i++){
            clk = 1; wait(1, SC_NS);
            clk = 0; wait(1, SC_NS);
        }

        sc_stop();
    }

    SC_CTOR(TB): cpu("cpu"){
        cpu.clk(clk);
        cpu.rst(rst);
        SC_THREAD(run);
    }
};
```

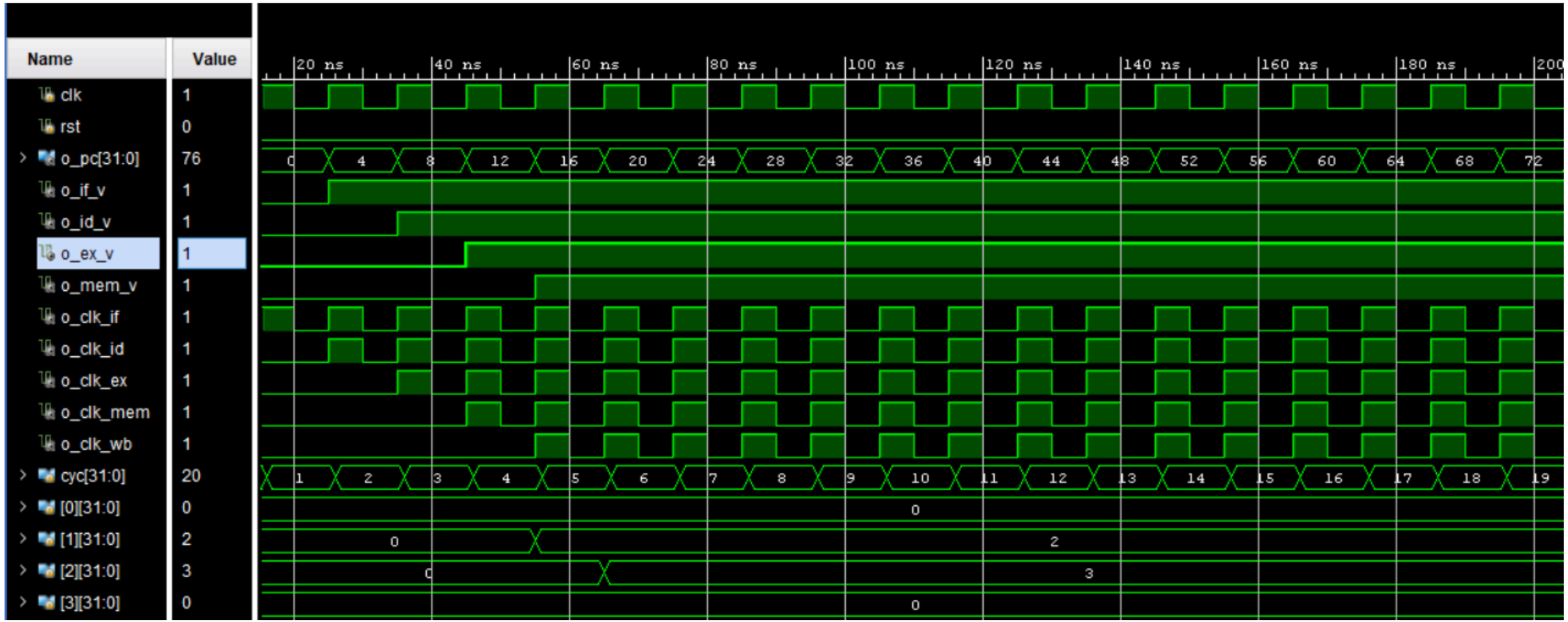
# Code: Main - with clock gating

```
// ================= MAIN =================
int sc_main(int argc, char* argv[]){
    TB tb("tb");

    sc_trace_file *tf = sc_create_vcd_trace_file("gated");
    tf->set_time_unit(1, SC_NS);

    sc_trace(tf, tb.cpu.sig_pc, "PC");
    sc_trace(tf, tb.cpu.sig_if, "IF");
    sc_trace(tf, tb.cpu.sig_id, "ID");
    sc_trace(tf, tb.cpu.sig_ex, "EX");
    sc_trace(tf, tb.cpu.sig_mem, "MEM");
    sc_trace(tf, tb.clk, "clk");
    sc_trace(tf, tb.rst, "rst");

    sc_start();

    sc_close_vcd_trace_file(tf);

    return 0;
}
```

# Output

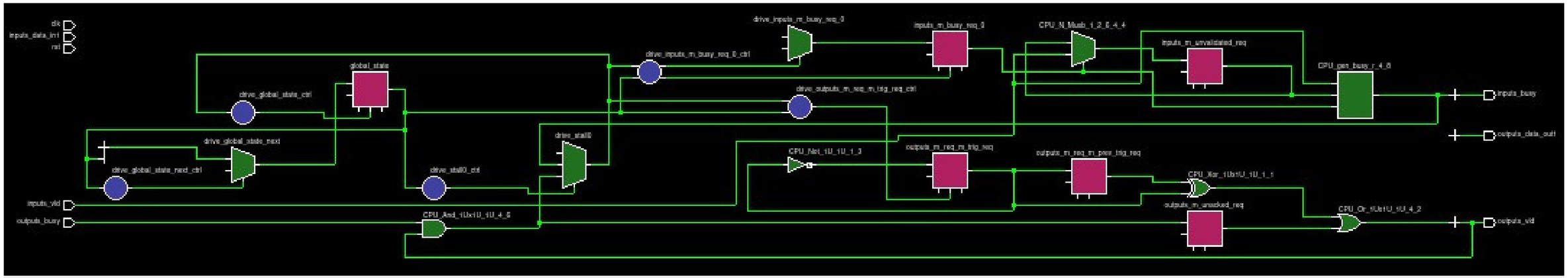
```
        SystemC 3.0.2-Accellera --- Apr 16 2026 08:48:44
        Copyright (c) 1996-2025 by all Contributors,
        ALL RIGHTS RESERVED

Info: (I703) tracing timescale unit set: 1 ns (gated.vcd)
PC=4 IF=1 ID=0 EX=0 MEM=0
PC=8 IF=1 ID=1 EX=0 MEM=0
PC=12 IF=1 ID=1 EX=1 MEM=0
PC=16 IF=1 ID=1 EX=1 MEM=1
PC=20 IF=1 ID=1 EX=1 MEM=1
PC=20 IF=0 ID=1 EX=1 MEM=1
PC=20 IF=0 ID=0 EX=1 MEM=1
PC=20 IF=0 ID=0 EX=0 MEM=1
PC=20 IF=0 ID=0 EX=0 MEM=0
```

# Reports


Name
Value
clk 1
rst 0
o_pc[31:0] 76
o_if_v 1
o_id_v 1
o_ex_v 1
o_mem_v 1
o_clk_if 1
o_clk_id 1
o_clk_ex 1
o_clk_mem 1
o_clk_wb 1
cyc[31:0] 20
[0][31:0] 0
[1][31:0] 2
[2][31:0] 3
[3][31:0] 0
20 ns
40 ns
60 ns
80 ns
100 ns
120 ns
140 ns
160 ns
180 ns

# Reports

## Schematic



# Reports

## Overall Metrics

### Area Metrics

| | |
|---|---|
| Combinational Area: | 47 |
| Resource Comb. Area: | 16 |
| Mux Comb. Area: | 14 |
| Other Comb. Area: | 17 |
| Sequential Area: | 47 |
| Black Box Area: | 0 |
| Total Bound Operations: | 17 |
| **Total Area:** | **94** |

### Resource Metrics

| | |
|---|---|
| Number of Instances: | 5 |
| Widest Input or Output: | 3 |
| Number of Pipelined Instances: | 0 |
| Bound Operations: | 5 |
| **Total Resource Area:** | **16** |

### Mux Metrics

| | |
|---|---|
| Number of Instances: | 4 |
| Widest Input or Output: | 2 |
| Largest Fanin: | 3 |
| Implicit Mux Area: | 12 |
| Explicit Mux Area: | 2 |
| Muxed Operations: | 10 |
| **Total Mux Area:** | **14** |

### Register Metrics

| | |
|---|---|
| Register Count: | 6 |
| Register Bits: | 7 |
| Bound Register Operations: | 12 |
| **Total Register Area:** | **56** |

### Memory Metrics

| | |
|---|---|
| Number of Instances: | 0 |
| Largest Memory (b): | 0 |
| Bound Memory Operations: | 0 |
| **Total Memory Area:** | **0** |

### Timing Metrics

| | |
|---|---|
| Time Units: | ns |
| Clock Period: | 20.00 |
| Cycle Slack: | 19.00 |
| Path Delay Limit: | 105 |
| Target Delay: | 1.05 |
| **Worst Slack:** | **-0.43** |

# Reports

```
##########################################################################
# Power Report
##########################################################################
Instance: /CPU
Power Unit: W
PDB Frames: /stim#1/frame#0
  ----------------------------------------------------------------------
    Category          Leakage     Internal    Switching        Total    Row%
  ----------------------------------------------------------------------
       memory      0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     register      5.85541e-10  5.50085e-07  8.05950e-09  5.58730e-07  83.98%
        latch      0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        logic      5.32648e-10  3.52580e-08  1.46880e-08  5.04787e-08   7.59%
         bbox      0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        clock      0.00000e+00  0.00000e+00  5.61330e-08  5.61330e-08   8.44%
          pad      0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
           pm      0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  ----------------------------------------------------------------------
    Subtotal      1.11819e-09  5.85343e-07  7.88805e-08  6.65341e-07 100.01%
  Percentage            0.17%       87.98%       11.86%      100.00% 100.00%
  ----------------------------------------------------------------------
```

# Code: Design - without clock gating

```cpp
#include <systemc.h>
#include <iostream>
#include <cstring>
using namespace std;

enum { MEM_WORDS=256, RF_SIZE=32 };

struct IF_ID  { uint32_t pc=0,inst=0; bool v=false; };
struct ID_EX  { uint32_t pc=0,a=0,b=0,imm=0; uint8_t rs1=0,rs2=0,rd=0; bool v=false; };
struct EX_MEM { uint32_t pc=0,alu=0; uint8_t rd=0; bool v=false; };
struct MEM_WB { uint32_t alu=0; uint8_t rd=0; bool v=false; };

SC_MODULE(CPU){
    sc_in<bool> clk, rst;

    uint32_t imem[MEM_WORDS];
    uint32_t rf[RF_SIZE];

    IF_ID if_id;
    ID_EX id_ex;
    EX_MEM ex_mem;
    MEM_WB mem_wb;

    uint32_t pc = 0;

    sc_signal<uint32_t> sig_pc;
    sc_signal<bool> sig_if, sig_id, sig_ex, sig_mem;

    uint32_t alu(uint32_t a, uint32_t b){
        return a + b;
    }

    void load_program(){
        memset(imem,0,sizeof imem);

        imem[0] = 0x00200093;
        imem[1] = 0x00300113;
        imem[2] = 0x002081B3;
        imem[3] = 0x00418213;
        imem[4] = 0x002202B3;
        imem[5] = 0x00000000;
    }
```

# Code: Design - without clock gating

```
void tick(){
    if(rst.read()){
        pc = 0;
        memset(rf,0,sizeof rf);
        if_id = {}; id_ex = {}; ex_mem = {}; mem_wb = {};
        return;
    }

    // 🔴 NORMAL VERSION = ALWAYS ACTIVE
    bool en_if  = true;
    bool en_id  = true;
    bool en_ex  = true;
    bool en_mem = true;
    bool en_wb  = true;

    // WB
    if(en_wb && mem_wb.rd){
        rf[mem_wb.rd] = mem_wb.alu;
    }

    // EX
    EX_MEM ex_mem_n = {};
    if(en_ex){
        uint32_t res = alu(id_ex.a, id_ex.b);
        ex_mem_n = {id_ex.pc, res, id_ex.rd, true};
    }

    // MEM
    MEM_WB mem_wb_n = {};
    if(en_mem){
        mem_wb_n = {ex_mem.alu, ex_mem.rd, true};
    }
```

```
    // ID
    ID_EX id_ex_n = {};
    if(en_id){
        uint32_t inst = if_id.inst;

        uint8_t opcode = inst & 0x7F;
        uint8_t rs1 = (inst >> 15) & 0x1F;
        uint8_t rs2 = (inst >> 20) & 0x1F;
        uint8_t rd  = (inst >> 7)  & 0x1F;
        int32_t imm = (int32_t)inst >> 20;

        uint32_t a = rf[rs1];
        uint32_t b = (opcode == 0x13) ? imm : rf[rs2];

        id_ex_n = {if_id.pc, a, b, (uint32_t)imm, rs1, rs2, rd, true};
    }

    // IF (ALWAYS FETCH — EVEN INVALID)
    IF_ID if_id_n = {};
    if(en_if){
        uint32_t instr = imem[pc >> 2];

        if_id_n = {pc, instr, true};  // ❗ no condition
        pc += 4;
    }

    if_id = if_id_n;
    id_ex = id_ex_n;
    ex_mem = ex_mem_n;
    mem_wb = mem_wb_n;

    sig_pc.write(pc);
    sig_if.write(if_id.v);
    sig_id.write(id_ex.v);
    sig_ex.write(ex_mem.v);
    sig_mem.write(mem_wb.v);
```

# Code: Design - without clock gating

```
        // PRINT
        cout << "PC=" << pc
             << " IF=" << if_id.v
             << " ID=" << id_ex.v
             << " EX=" << ex_mem.v
             << " MEM=" << mem_wb.v
             << endl;
    }

    SC_CTOR(CPU){
        load_program();
        SC_METHOD(tick);
        sensitive << clk.pos();
    }
};
```

# Code: Testbench - without clock gating

```
SC_MODULE(TB){
    sc_signal<bool> clk, rst;
    CPU cpu;

    void run(){
        rst = 1; clk = 0; wait(1, SC_NS);
        rst = 0;

        for(int i=0;i<15;i++){
            clk = 1; wait(1, SC_NS);
            clk = 0; wait(1, SC_NS);
        }

        sc_stop();
    }

    SC_CTOR(TB): cpu("cpu"){
        cpu.clk(clk);
        cpu.rst(rst);
        SC_THREAD(run);
    }
};
```

# Code: Main - without clock gating

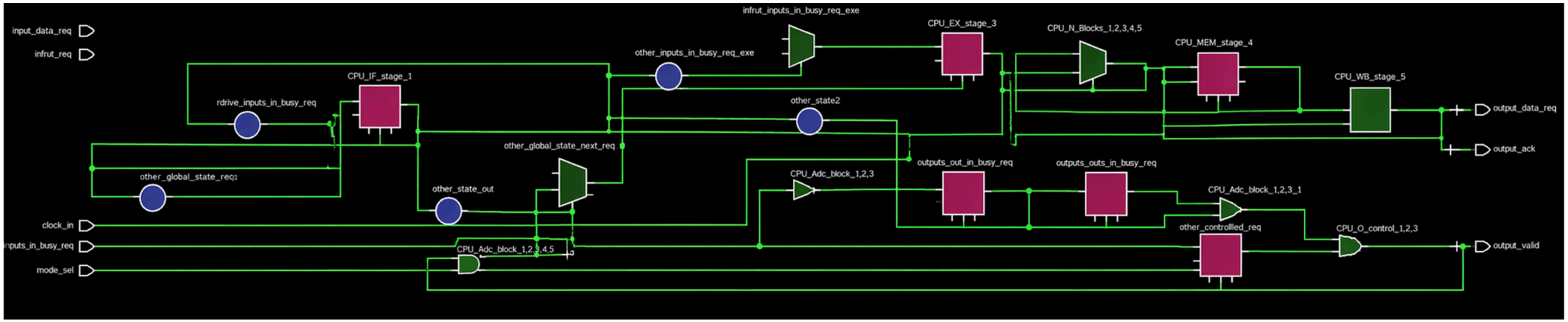
```
int sc_main(int argc, char* argv[]){
    TB tb("tb");

    sc_trace_file *tf = sc_create_vcd_trace_file("normal");

    sc_trace(tf, tb.cpu.sig_pc, "PC");
    sc_trace(tf, tb.cpu.sig_if, "IF");
    sc_trace(tf, tb.cpu.sig_id, "ID");
    sc_trace(tf, tb.cpu.sig_ex, "EX");
    sc_trace(tf, tb.cpu.sig_mem, "MEM");

    sc_start();
    sc_close_vcd_trace_file(tf);

    return 0;
}
```

# Output

```
        SystemC 3.0.2-Accellera --- Apr 16 2026 07:56:13
        Copyright (c) 1996-2025 by all Contributors,
        ALL RIGHTS RESERVED

Info: (I702) default timescale unit used for tracing: 1 ps (no_cg.vcd)
PC=4  IF=1 ID=1 EX=1 MEM=1
PC=8  IF=1 ID=1 EX=1 MEM=1
PC=12 IF=1 ID=1 EX=1 MEM=1
PC=16 IF=1 ID=1 EX=1 MEM=1
PC=20 IF=1 ID=1 EX=1 MEM=1
PC=24 IF=1 ID=1 EX=1 MEM=1
PC=28 IF=1 ID=1 EX=1 MEM=1
PC=32 IF=1 ID=1 EX=1 MEM=1
PC=36 IF=1 ID=1 EX=1 MEM=1
PC=40 IF=1 ID=1 EX=1 MEM=1
PC=44 IF=1 ID=1 EX=1 MEM=1
PC=48 IF=1 ID=1 EX=1 MEM=1
PC=52 IF=1 ID=1 EX=1 MEM=1
PC=56 IF=1 ID=1 EX=1 MEM=1
PC=60 IF=1 ID=1 EX=1 MEM=1
PC=64 IF=1 ID=1 EX=1 MEM=1
PC=68 IF=1 ID=1 EX=1 MEM=1
PC=72 IF=1 ID=1 EX=1 MEM=1
PC=76 IF=1 ID=1 EX=1 MEM=1
PC=80 IF=1 ID=1 EX=1 MEM=1

Info: /OSCI/SystemC: Simulation stopped by user.
(base) anaghasarswathy@anaghas-MacBook-Air-3 riscv_final_project % nano cpu_no_cg.cpp
(base) anaghasarswathy@anaghas-MacBook-Air-3 riscv_final_project % g++ -std=c++17 cpu_no_cg.cpp -o no_cg $(pkg-config --cflags --libs systemc)
./no_cg

        SystemC 3.0.2-Accellera --- Apr 16 2026 07:56:13
        Copyright (c) 1996-2025 by all Contributors,
        ALL RIGHTS RESERVED

PC=4  IF=1 ID=1 EX=1 MEM=1

Info: (I702) default timescale unit used for tracing: 1 ps (no_cg.vcd)
PC=8  IF=1 ID=1 EX=1 MEM=1
PC=12 IF=1 ID=1 EX=1 MEM=1
PC=16 IF=1 ID=1 EX=1 MEM=1
PC=20 IF=1 ID=1 EX=1 MEM=1
PC=24 IF=1 ID=1 EX=1 MEM=1
PC=28 IF=1 ID=1 EX=1 MEM=1
PC=32 IF=1 ID=1 EX=1 MEM=1
PC=36 IF=1 ID=1 EX=1 MEM=1
PC=40 IF=1 ID=1 EX=1 MEM=1
PC=44 IF=1 ID=1 EX=1 MEM=1
PC=48 IF=1 ID=1 EX=1 MEM=1
PC=52 IF=1 ID=1 EX=1 MEM=1
PC=56 IF=1 ID=1 EX=1 MEM=1
PC=60 IF=1 ID=1 EX=1 MEM=1
PC=64 IF=1 ID=1 EX=1 MEM=1
```

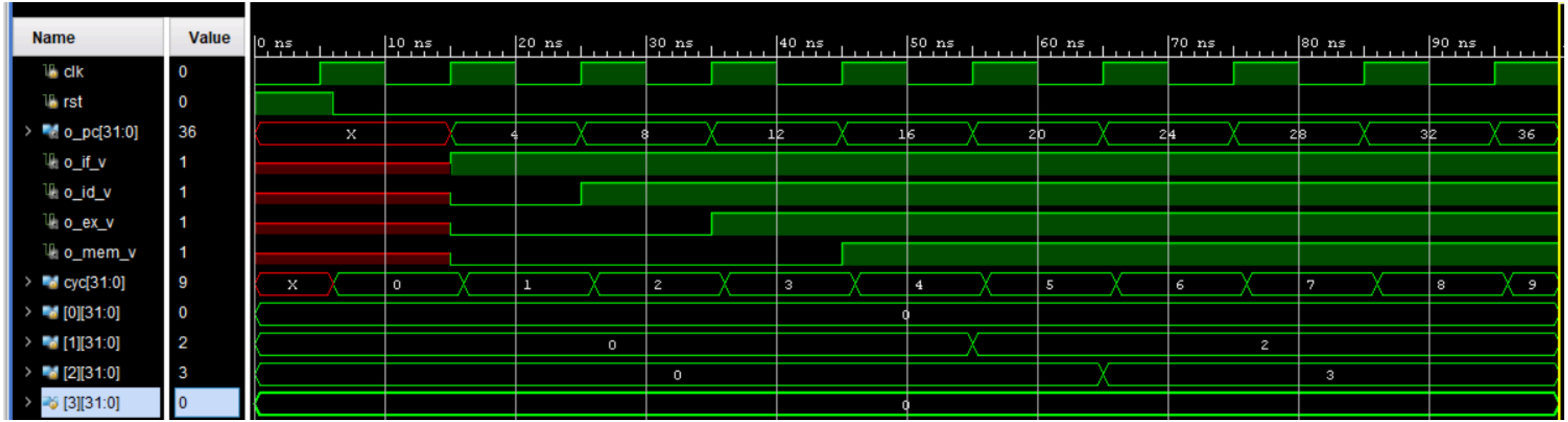
# Output

## Schematic



# Output

## Simulation



# Report:

## Overall Metrics

| Area Metrics | |
|---|---|
| Combinational Area: | 52 |
| Resource Comb. Area: | 18 |
| Mux Comb. Area: | 11 |
| Other Comb. Area: | 23 |
| Sequential Area: | 74 |
| Black Box Area: | 0 |
| Total Bound Operations: | 17 |
| **Total Area:** | **126** |

| Resource Metrics | |
|---|---|
| Number of Instances: | 5 |
| Widest Input or Output: | 3 |
| Number of Pipelined Instances: | 0 |
| Bound Operations: | 5 |
| **Total Resource Area:** | **16** |

| Mux Metrics | |
|---|---|
| Number of Instances: | 3 |
| Widest Input or Output: | 2 |
| Largest Fanin: | 2 |
| Implicit Mux Area: | 8 |
| Explicit Mux Area: | 3 |
| Muxed Operations: | 7 |
| **Total Mux Area:** | **11** |

| Register Metrics | |
|---|---|
| Register Count: | 6 |
| Register Bits: | 12 |
| Bound Register Operations: | 12 |
| **Total Register Area:** | **74** |

| Memory Metrics | |
|---|---|
| Number of Instances: | 0 |
| Largest Memory (b): | 0 |
| Bound Memory Operations: | 0 |
| **Total Memory Area:** | **0** |

| Timing Metrics | |
|---|---|
| Time Units: | ns |
| Clock Period: | 20.00 |
| Cycle Slack: | 19.00 |
| Path Delay Limit: | 105 |
| Target Delay: | 1.05 |
| **Worst Slack:** | **-0.38** |

# Reports

```
#################################################################
# Clocks Report
#################################################################
-----------------------------------------------------------------
                                    | /stim#1/frame#0 |    clock
  sdc_freq  gates  flops  icgcs   area |        sim_freq | sdc_name
-----------------------------------------------------------------
  5.000e+07    31      7      0   71.1 |       5.000e+07 |      clk
-----------------------------------------------------------------
Normal exit.
```

# Reports

```
#######################################################################
# Power Report
#######################################################################
Instance: /CPU
Power Unit: W
PDB Frames: /stim#1/frame#0
-----------------------------------------------------------------------------
  Category         Leakage       Internal      Switching          Total     Row%
-----------------------------------------------------------------------------
    memory    0.00000e+00    0.00000e+00    0.00000e+00    0.00000e+00    0.00%
  register    5.85541e-10    8.05950e-07    1.62300e-07    9.69841e-07   83.98%
     latch    0.00000e+00    0.00000e+00    0.00000e+00    0.00000e+00    0.00%
     logic    5.32648e-10    3.52580e-08    1.46880e-08    5.04787e-08    7.59%
      bbox    0.00000e+00    0.00000e+00    0.00000e+00    0.00000e+00    0.00%
     clock    0.00000e+00    0.00000e+00    1.13870e-07    1.13870e-07   16.44%
       pad    0.00000e+00    0.00000e+00    0.00000e+00    0.00000e+00    0.00%
        pm    0.00000e+00    0.00000e+00    0.00000e+00    0.00000e+00    0.00%
-----------------------------------------------------------------------------
  Subtotal    1.11819e-09    8.41545e-07    2.90050e-07    1.13342e-06  100.01%
Percentage          0.10%        74.26%          25.64%        100.00%  100.00%
-----------------------------------------------------------------------------
```

# Thank You